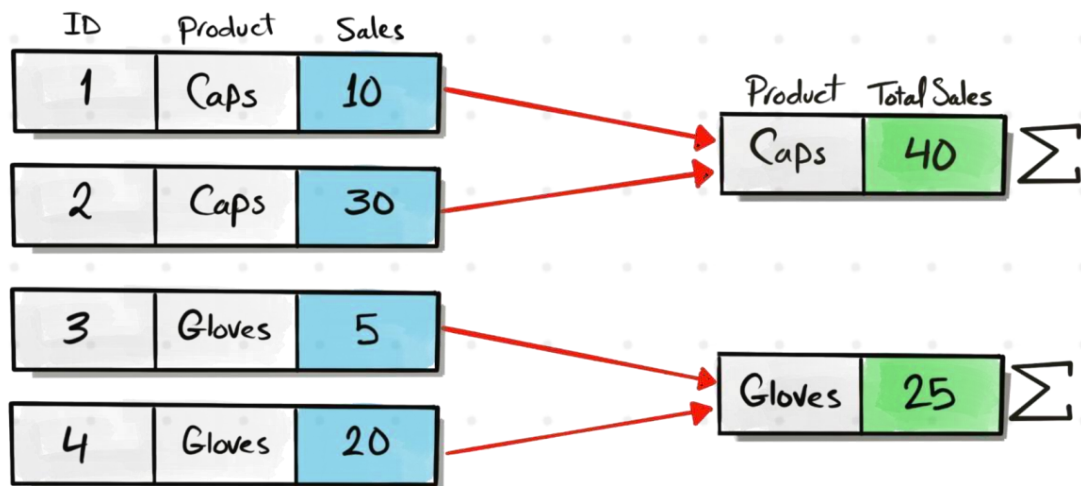


WINDOWS FUNCTIONS

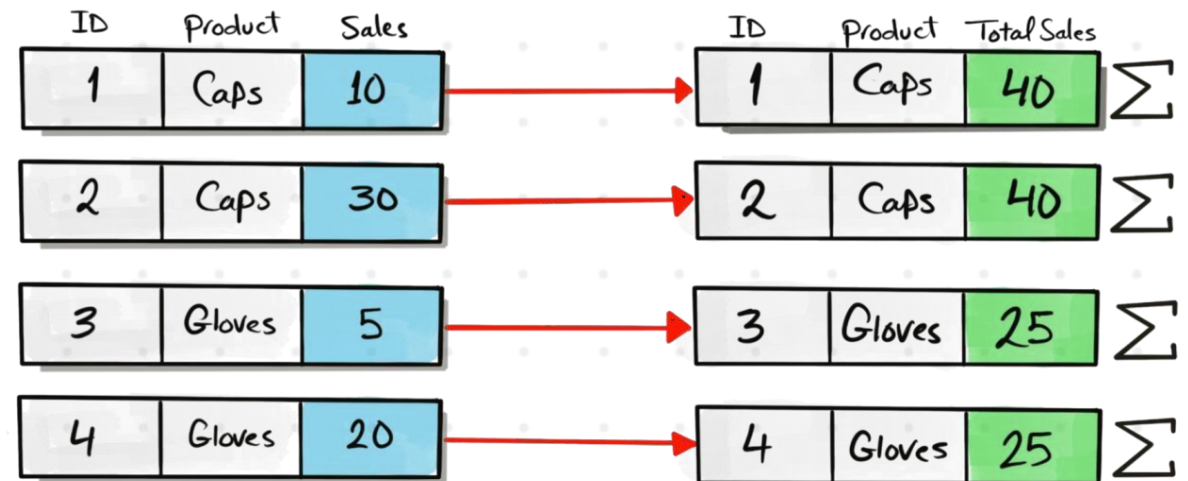
Group By

GROUP BY



It **collapse** groups of rows into a single row (Group-Level-Calculations)

Window Functions



It **doesn't collapse** rows into a single row (Row-Level-Calculations)

GROUP BY

Functions

Aggregate Functions

COUNT (*expr*)

SUM (*expr*)

MAX (*expr*)

MIN (*expr*)

MIN (*expr*)

WINDOW

Functions

Aggregate Functions

COUNT (*expr*)

SUM (*expr*)

MAX (*expr*)

MIN (*expr*)

MIN (*expr*)

Rank Functions

ROW_NUMBER ()

RANK ()

DENSE_RANK ()

CUME_DIST ()

PERCENT_RANK ()

NTILE (*n*)

Value (Analytics) Functions

LEAD (*expr, offset, default*)

LAG (*expr, offset, default*)

FIRST_VALUE (*expr*)

FIRST_VALUE (*expr*)

GROUP BY

Simple Data Analysis
(Aggregations)

WINDOW

Advanced Data Analysis
(Aggregations + Details)

Window syntax

Over Clause

Window
Function

Partition
Clause

Order
Clause

Frame
Clause

AVG (Sales) OVER (PARTITION BY Category ORDER BY OrderDate ROWS UNBOUNDED PRECEDING)

Window syntax

Calculation used
on the Window

Window
Function

AVG (Sales) OVER (PARTITION BY Category ORDER BY OrderDate ROWS UNBOUNDED PRECEDING)

Function
Expression

Window Expressions

Empty

`RANK() OVER (ORDER BY OrderDate)`

Column

`AVG(Sales) OVER (ORDER BY OrderDate)`

Number

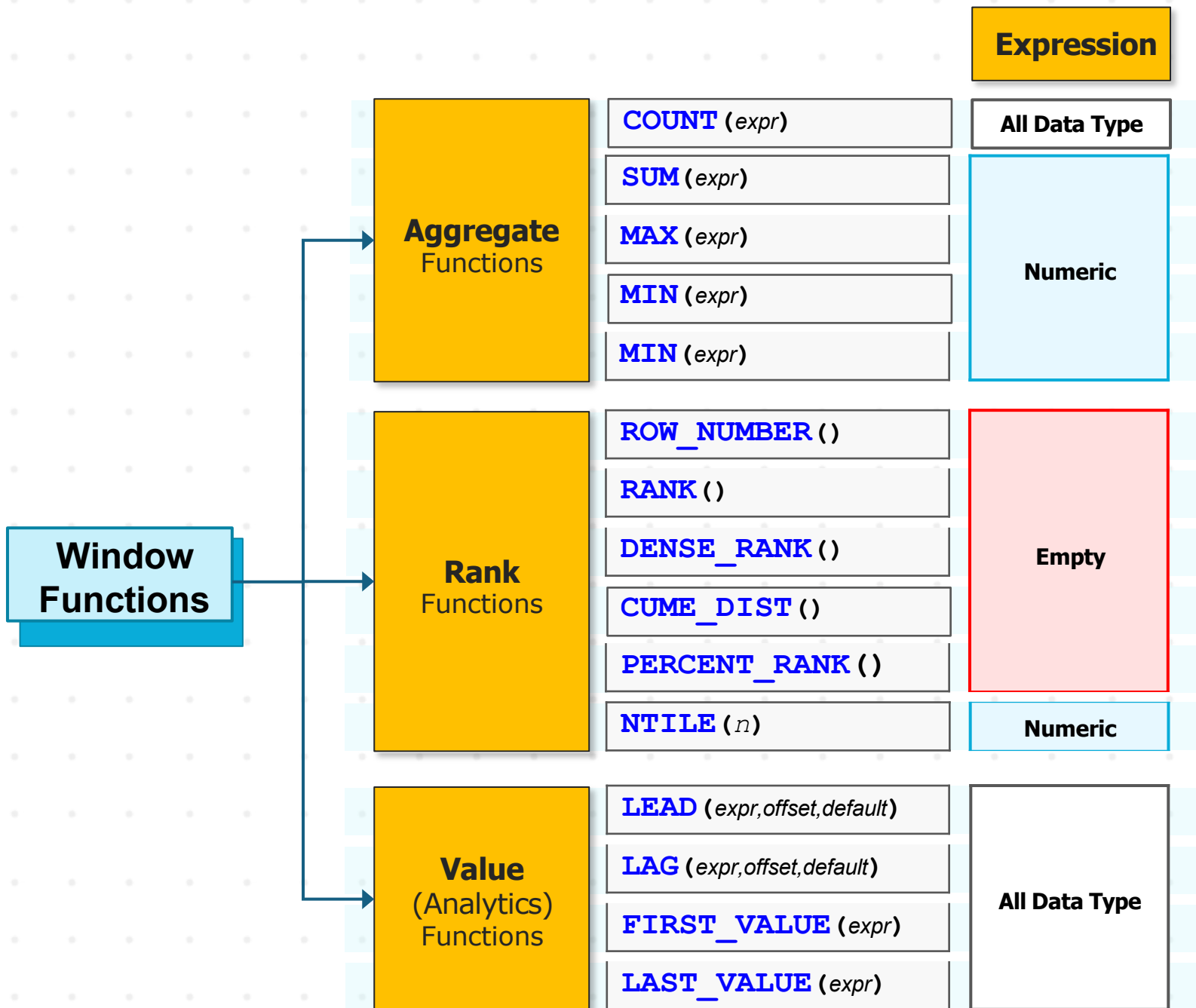
`NTEIL(2) OVER (ORDER BY OrderDate)`

Multiple Arguments

`LEAD(Sales,2,10) OVER (ORDER BY OrderDate)`

Conditional Logic

`SUM(CASE WHEN Sales > 100 THEN 1 ELSE 0 END) OVER (ORDER BY OrderDate)`



Window syntax

Calculation used
on the Window

Window
Function

Define the
Window

Over
Clause

AVG (Sales) **OVER** (**PARTITION BY** Category **ORDER BY** OrderDate **ROWS UNBOUNDED PRECEDING**)

Function
Expression

Partition
Clause

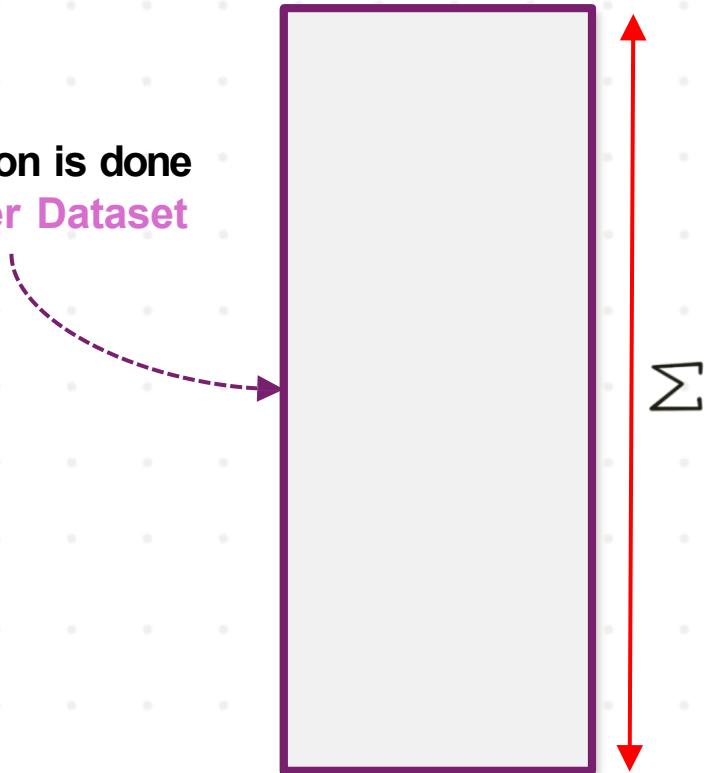
Divides the dataset into
windows (Partitions)

Partition By

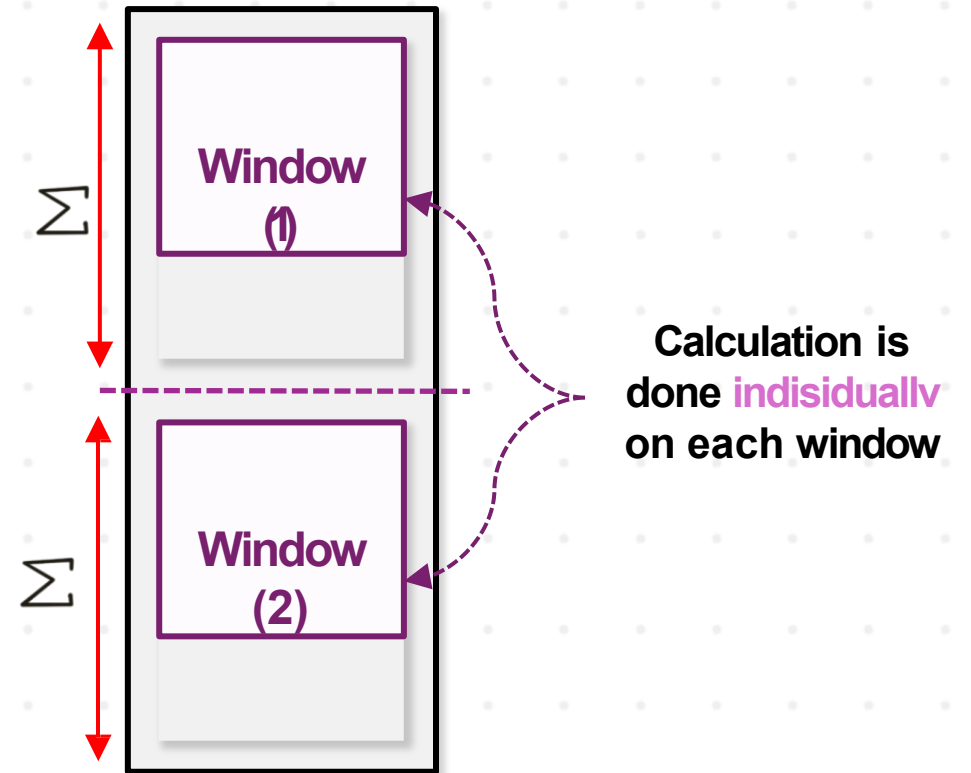
PARTITION BY divides the rows into groups, based on the column/s

```
SUM(Sales) OVER()
```

Calculation is done
on **entier Dataset**



```
SUM(Sales) OVER(PARTITION BY Product)
```



Partition By

PARTITION BY divides the rows into groups, based on the column/s

**Without
Partition By**

Total sales across all rows (Entire Result Set)

```
SUM(Sales) OVER ()
```

**Partition By
Single Column**

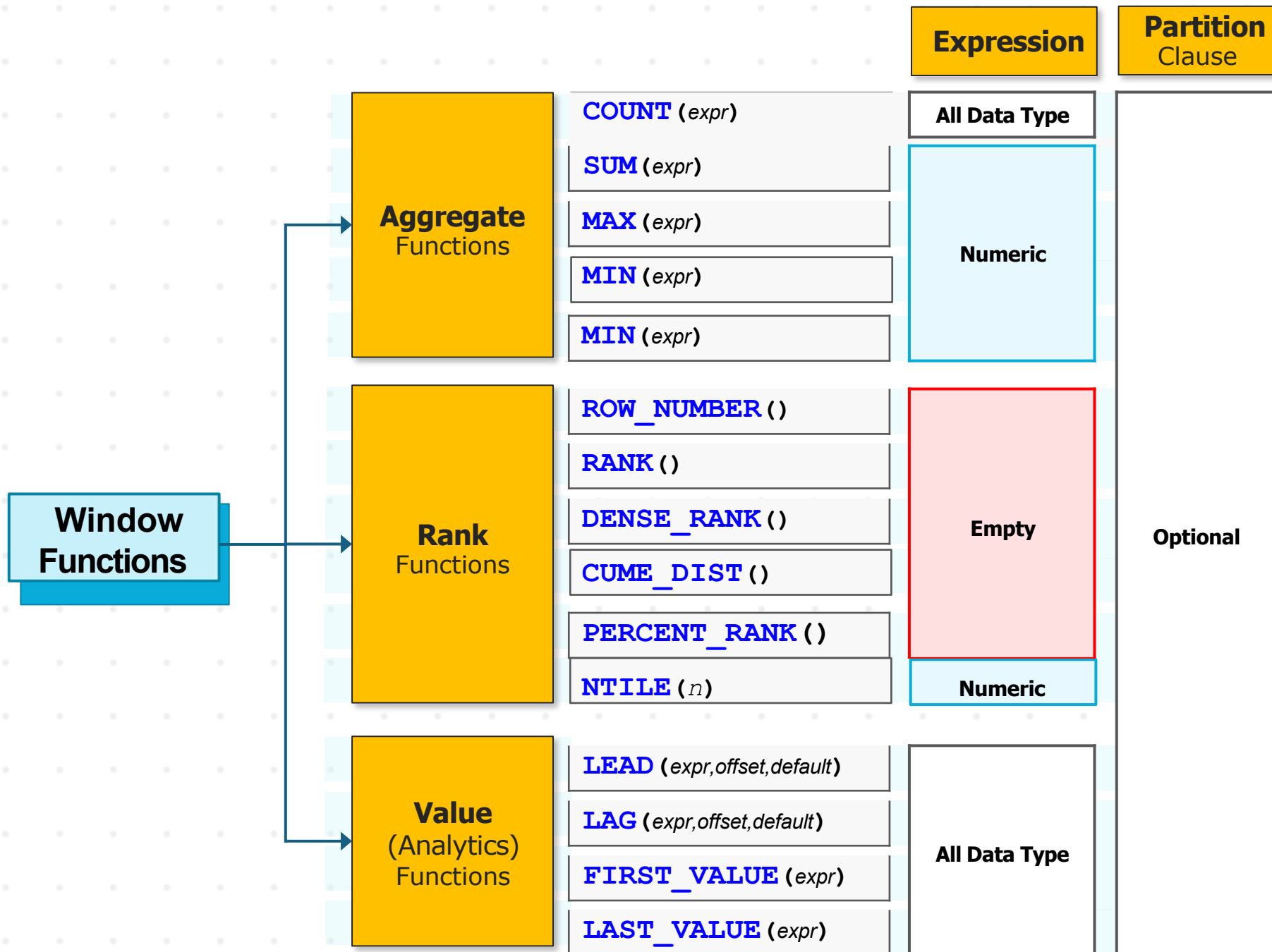
Total sales for each Product

```
SUM(Sales) OVER (PARTITION BY Product)
```

**Partition By
Combined-Columns**

Total sales for each combination of Product and Order Status

```
SUM(Sales) OVER (PARTITION BY Product, OrderStatus)
```



Window syntax

Calculation used
on the Window

Define the
Window

Window
Function

Over
Clause

AVG (Sales) **OVER** (**PARTITION BY** Category **ORDER BY** OrderDate ROWS UNBOUNDED PRECEDING)

Function
Expression

Partition
Clause

Order
Clause

Divides the dataset into
windows (Partitions)

sort the data
in a window

		Expression	Partition Clause	Order Clause
Window Functions	Aggregate Functions	COUNT (expr) SUM (expr) MAX (expr) MIN (expr) MIN (expr)	All Data Type	Optional
	Rank Functions	ROW_NUMBER () RANK () DENSE_RANK () CUME_DIST () PERCENT_RANK () NTILE (n)	Empty	Required
	Value (Analytics) Functions	LEAD (expr,offset,default) LAG (expr,offset,default) FIRST_VALUE (expr) LAST_VALUE (expr)	All Data Type	Required
		Numeric	Optional	Required
		Numeric		

Window syntax

Calculation used
on the Window

Define the
Window

Window
Function

Over
Clause

AVG (Sales) **OVER** (**PARTITION BY** Category **ORDER BY** OrderDate **ROWS UNBOUNDED PRECEDING**)

Function
Expression

Partition
Clause

Order
Clause

Frame
Clause

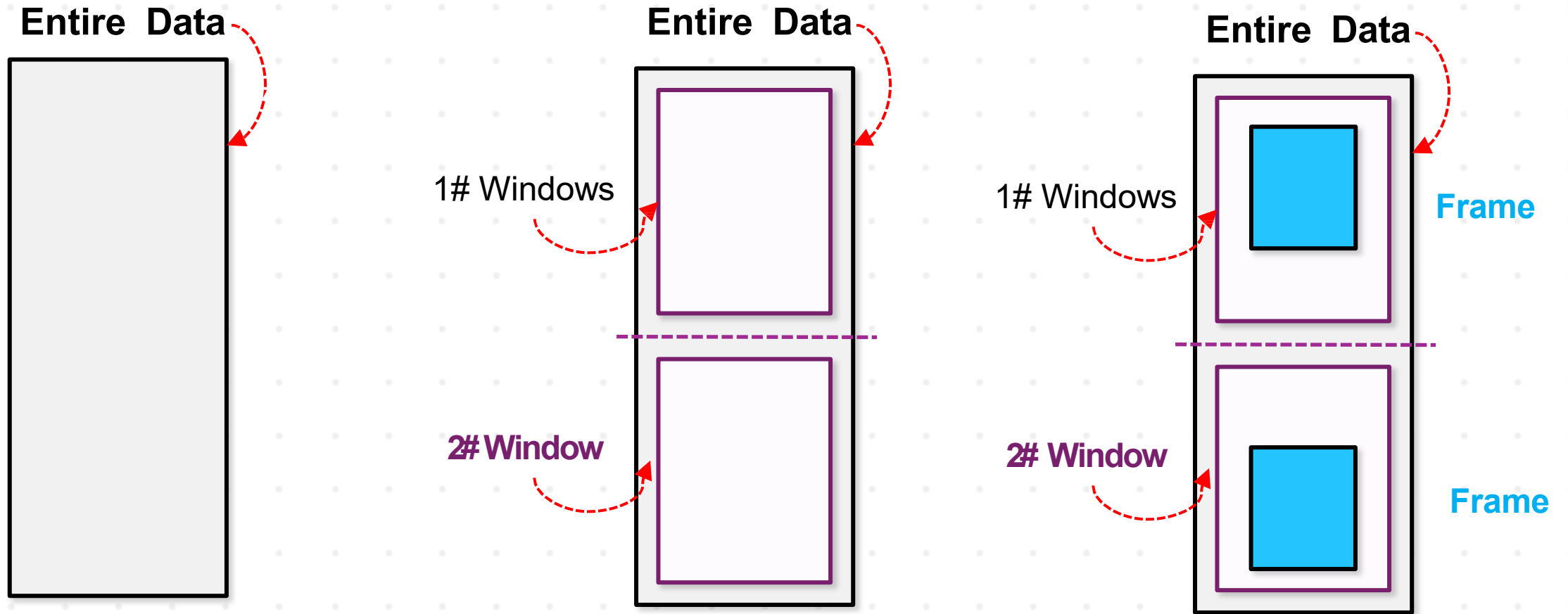
Divides the dataset into
windows (Partitions)

sort the
data in a
window

Define a subset of
rows in a window



Frame Clause



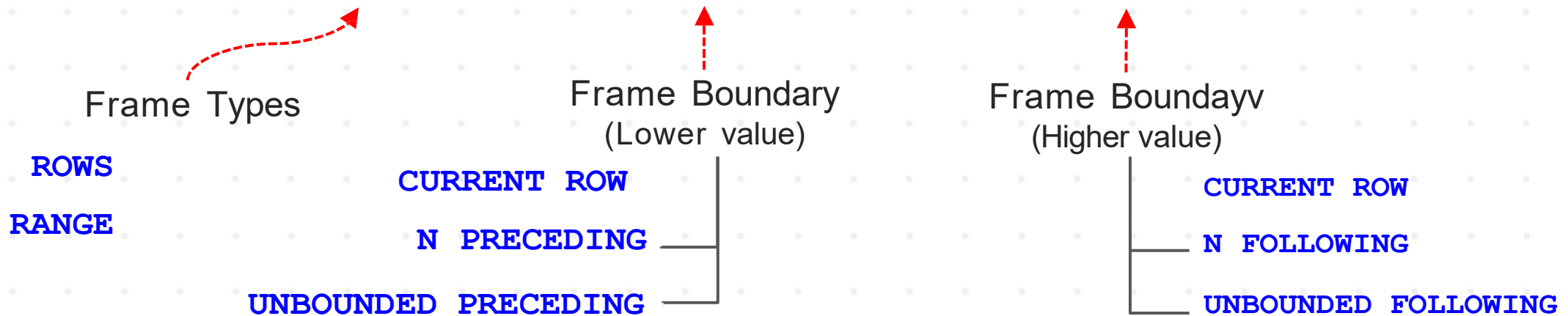
Window Functions

		Expression	Partition Clause	Order Clause	Frame Clause
Aggregate Functions	COUNT (expr)	All Data Type			
	SUM (expr)	Numeric			
	AVG (expr)				
	MAX (expr)				
	MIN (expr)				
Rank Functions	ROW_NUMBER ()	Empty	Optional	Required	Not allowed
	RANK ()				
	DENSE_RANK ()				
	CUME_DIST ()				
	PERCENT_RANK ()				
	NTILE (n)	Numeric			
Value (Analytics) Functions	LEAD (expr,offset,default)	All Data Type		Required	Not allowed
	LAG (expr,offset,default)				Optional
	FIRST_VALUE (expr)				
	LAST_VALUE (expr)				Should be used

Frame

`AVG(Sales) OVER (PARTITION BY Category ORDER BY OrderDate`

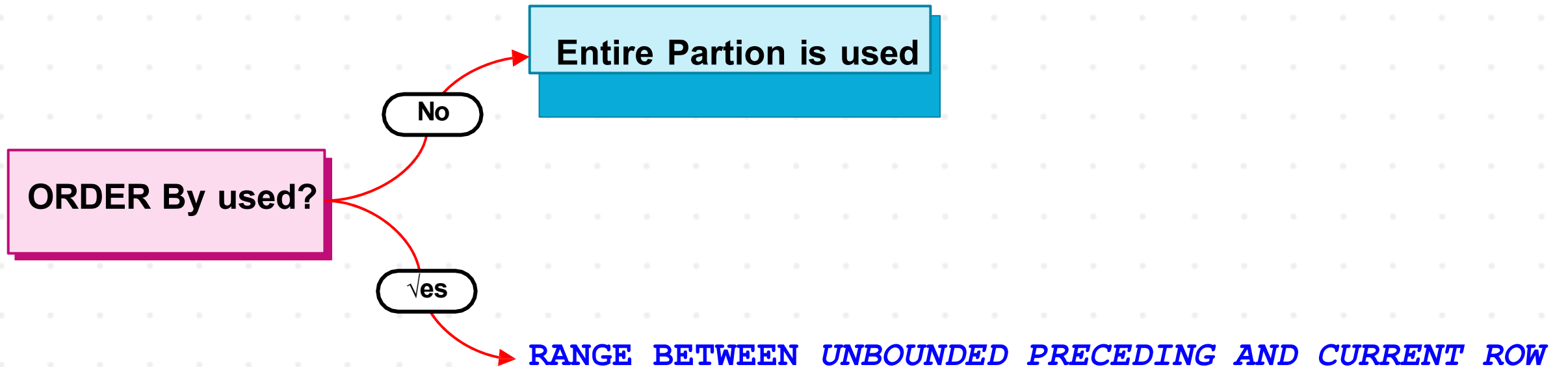
ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING)



Rules

- Frame Clause can only be used **together** with order by clause.
- Lower Value must be **BEFORE** the higher Value.

Frame



Window Rules

#1 RULE

Window functions can be used ONLY
in **SELECT** and **ORDER BY** Clauses

#2 RULE

Nesting Window Functions is not allowed !

#3 RULE

SQL **execute** WINDOW Functions **after** WHERE Clause

#4 RULE

Window Function can be used together with GROUP BY
in the same query, ONLY if the same **columns** are used

Sum Function

`SUM(Sales) OVER(PARTITION BY Product)`

Product	Sales	SUM
Caps	20	35
Caps	10	35
Caps	5	35
Gloves	30	100
Gloves	70	100
Gloves	NULL	100

$$20 + 10 + 5 = 35$$

$$30 + 70 = 100$$

Overall Total

`SUM(Sales) OVER()`

Overview of entire data

Month	Sales	SUM
Caps	20	175
Caps	10	175
Caps	30	175
Gloves	5	175
Gloves	70	175
Gloves	40	175

Total Per Groups

`SUM(Sales) OVER(
PARTITION BY Product)`

Compare Categories

Month	Sales	SUM
Caps	20	60
Caps	10	60
Caps	30	60
Gloves	5	105
Gloves	70	105
Gloves	40	105

Running Total

`SUM(Sales) OVER(
ORDER BY MONTH)`

progress over time

Month	Sales	SUM
Jan	20	20
Feb	10	30
Mar	30	60
Apr	5	65
Jun	70	135
Jul	40	175

Rolling Total

`SUM(Sales) OVER(
ORDER BY MONTH
ROWS 2 PRECEDING)`

progress over time in
specific fixed window

Month	Sales	SUM
Jan	20	20
Feb	10	30
Mar	30	60
Apr	5	45
Jun	70	105
Jul	40	105

Ranking Function

RANK () OVER (PARTITION BY ProductID ORDER BY Sales)

↑
Expression
must be **empty**

Partition By
Is **Optional**

↑
Order By
Is **required**

		Expression	Partition Clause	Order Clause	Frame Clause
Rank Functions	ROW_NUMBER ()	Empty	Optional	Required	Not allowed
	RANK ()				
	DENSE_RANK ()				
	CUME_DIST ()				
	PERCENT_RANK ()				
	NTILE (n)	Number			

RANK

Assign a **rank** to each row within a window

```
RANK () OVER (ORDER BY Sales DESC)
```

sales	Rank
100	1
80	2
80	2
50	4
20	5

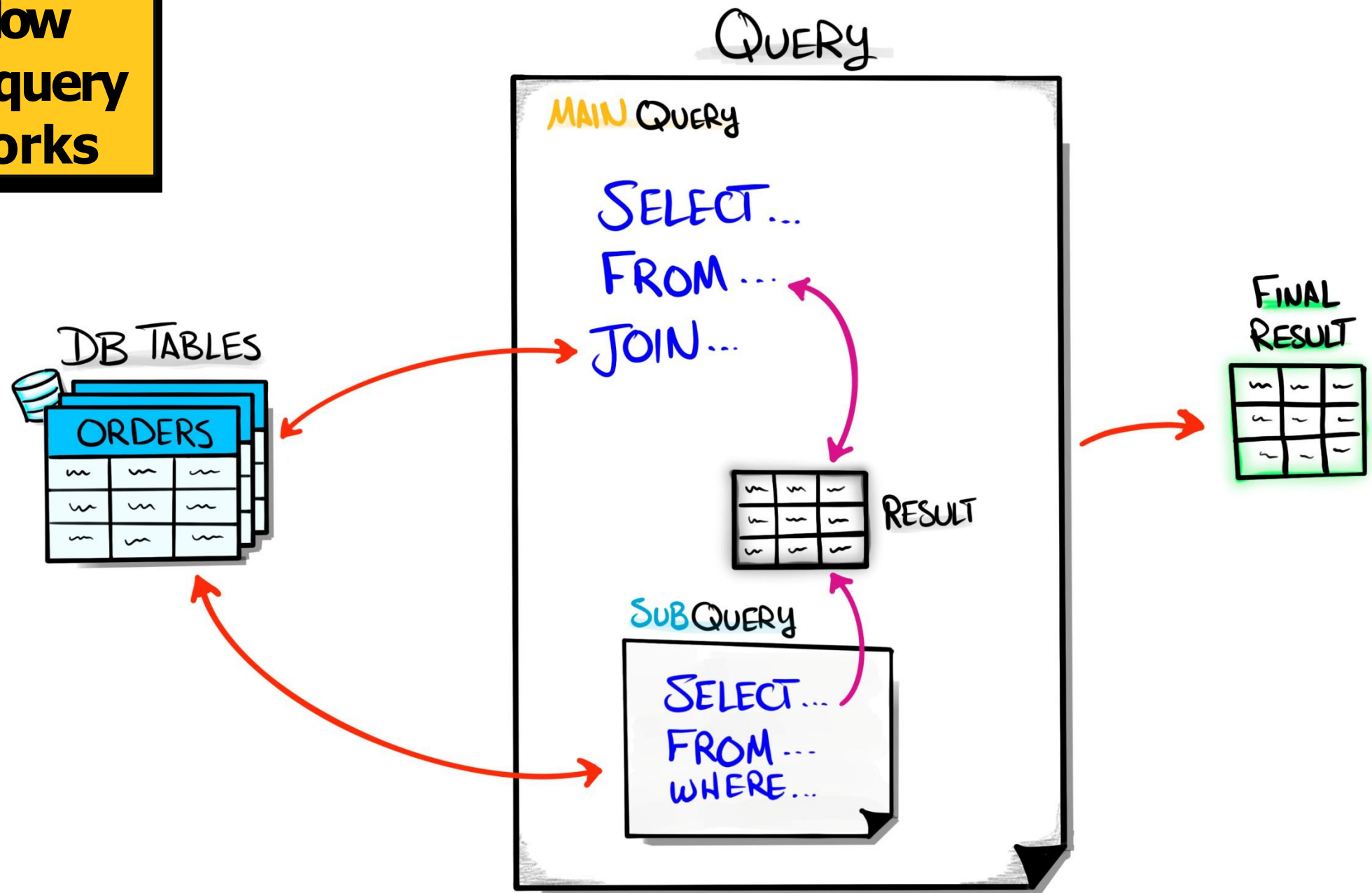
We have a **tie** here!

RANK () assigns a **same rank for** both of them

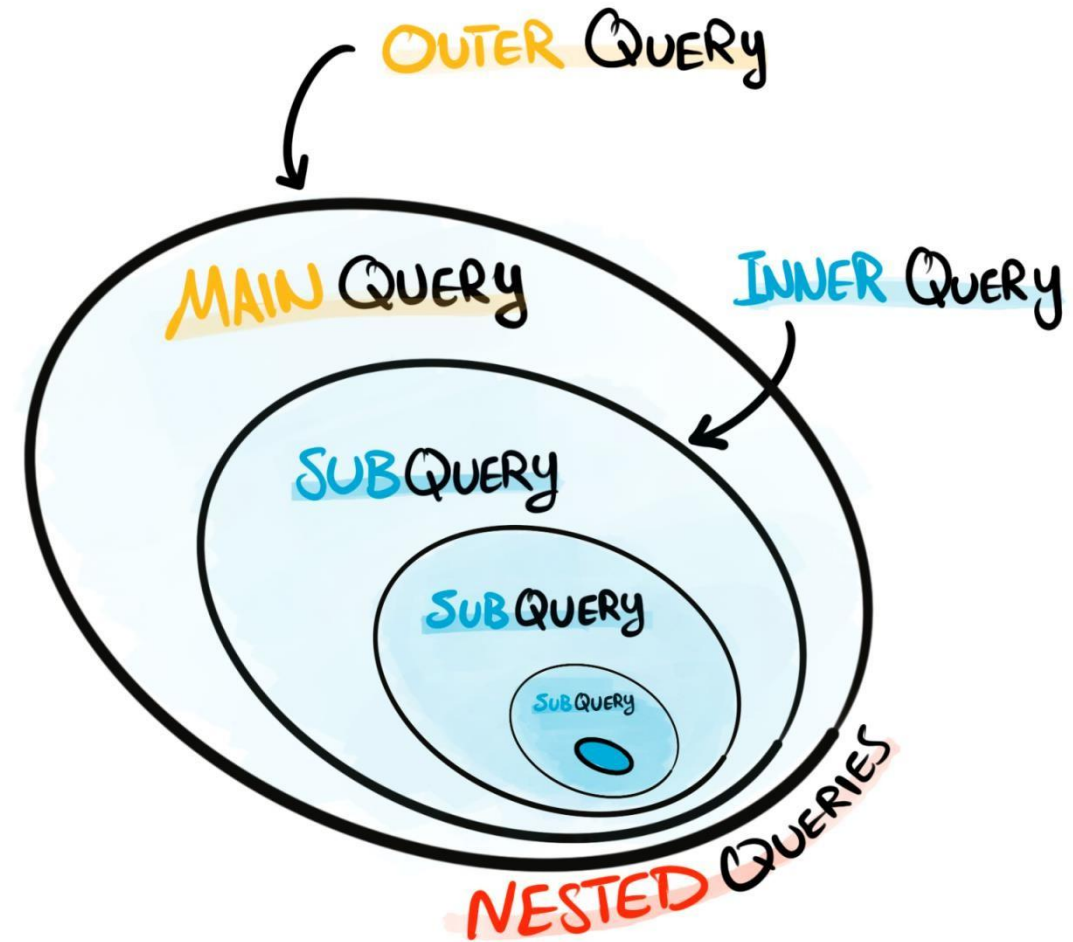
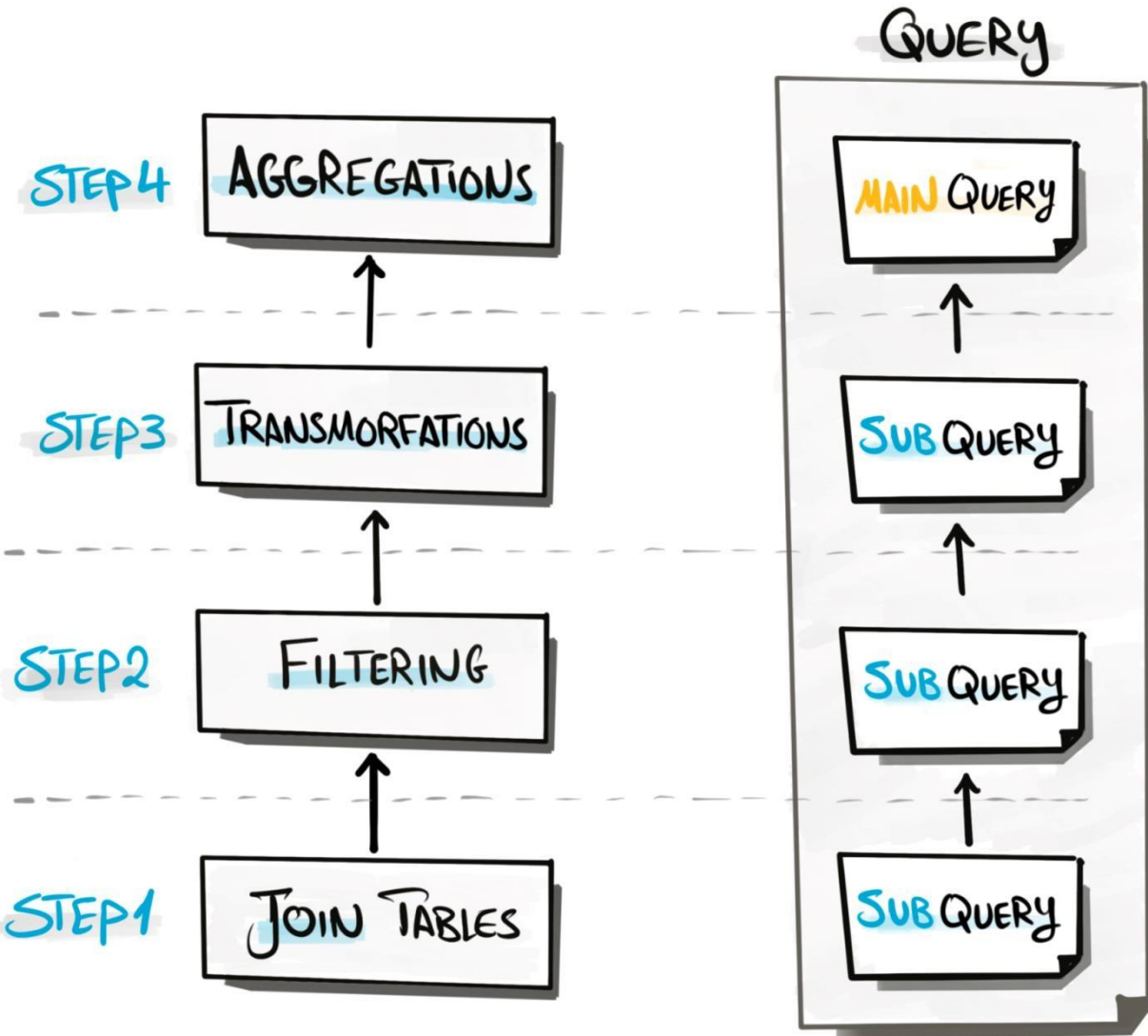
RANK () leaves a **GAP in Ranking** after a tie

SUBQUERIES

How subquery Works



SUBQUERY





DATABASE

ORDERS		
~	~	~
~	~	~
~	~	~

CUSTOMERS		
~	~	~
~	~	~
~	~	~

Subquery

SELECT...
FROM...
WHERE...

Result

~	~	~
~	~	~
~	~	~

Subquery

SELECT...
FROM...
WHERE...

Result

~	~	~
~	~	~
~	~	~

Subquery

SELECT...
FROM...
WHERE...

Result

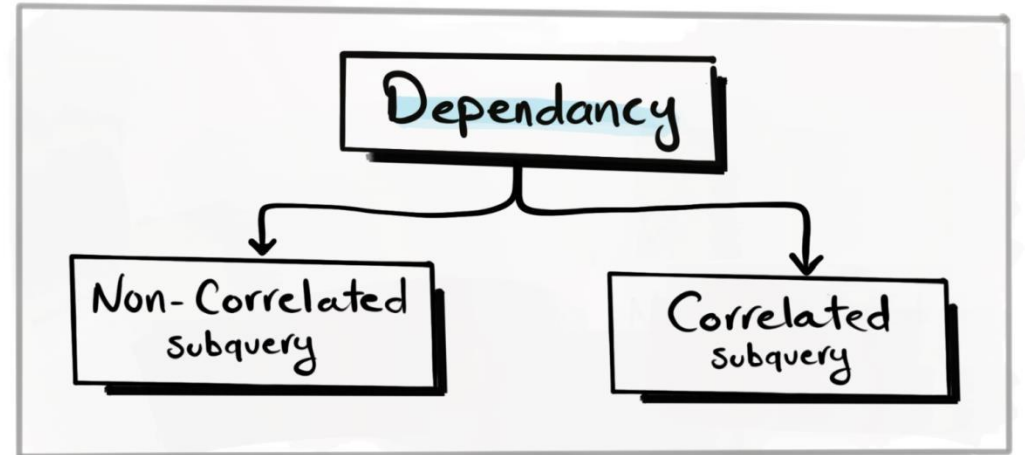
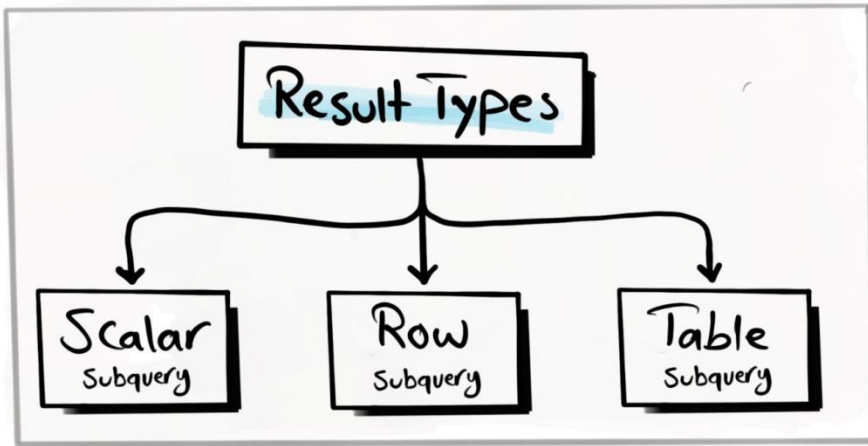
~	~	~
~	~	~
~	~	~

Final Result

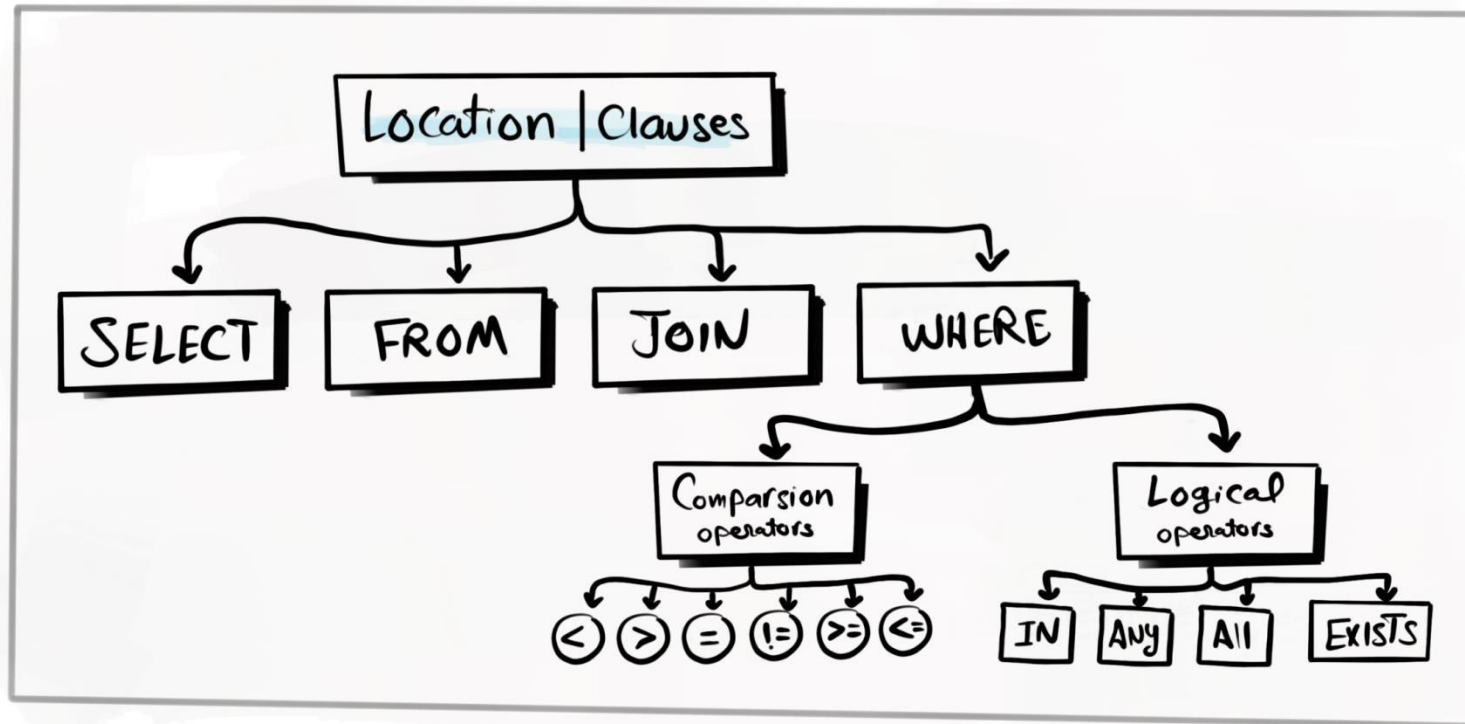
~	~	~
~	~	~
~	~	~

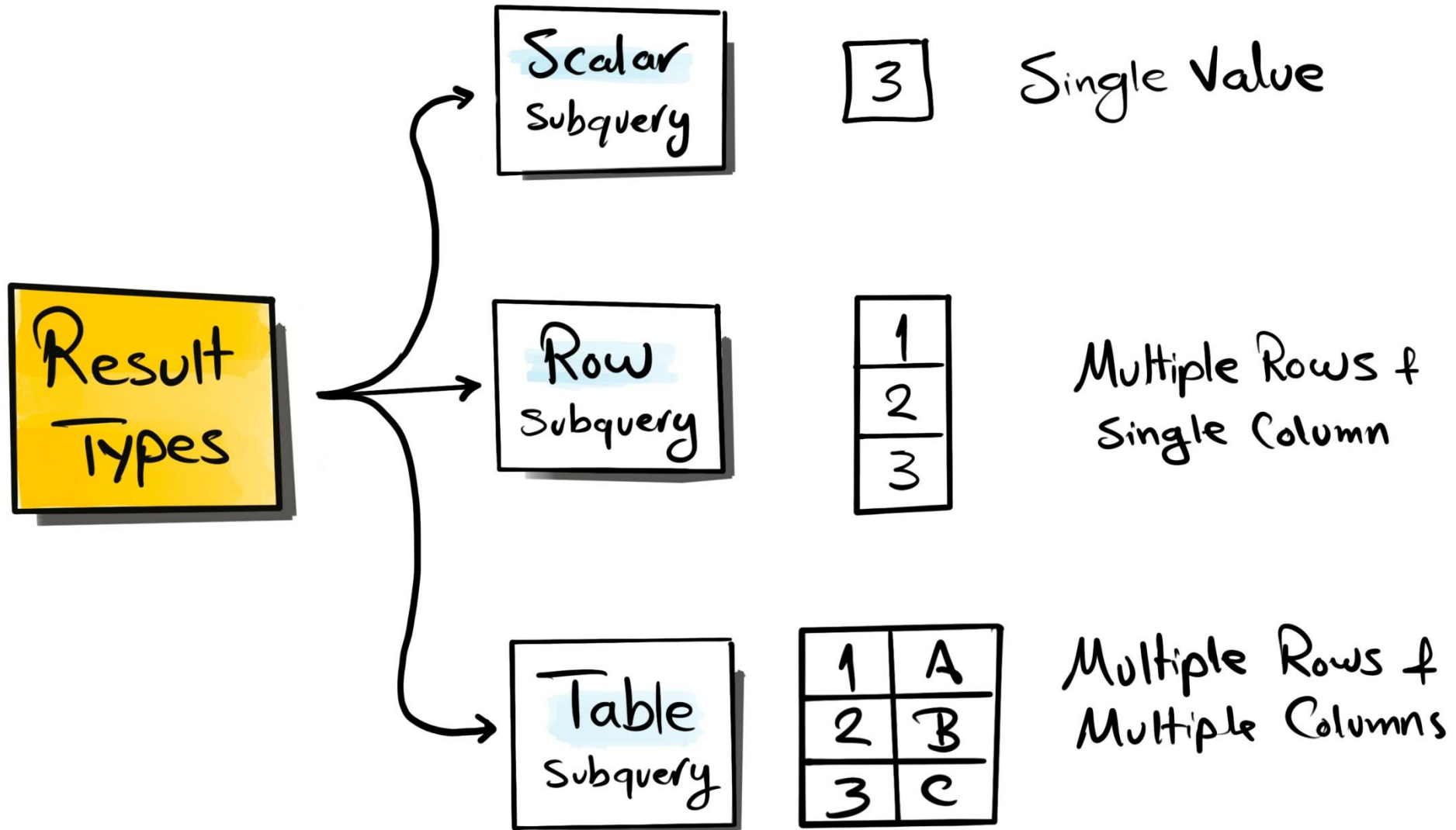
MAIN Query

SELECT...
FROM...
WHERE...



SUBQUERY





subquery in **FROM Clause**

Main Query

```
SELECT column1, column2,...  
FROM ( SELECT column FROM table1 WHERE condition ) AS alias
```

subquery

Orders

OrderID	CustomerID	Sales
1	2	10
2	3	15
3	1	20
4	1	60
5	2	25
6	3	50
7	1	60
8	4	90
9	2	20
10	3	60

Original Query

```
SELECT
*,
RANK() OVER (ORDER BY TotalSales DESC) CustomerRank
FROM
(SELECT
CustomerID,
SUM(Sales) TotalSales
FROM Sales.Orders
GROUP BY CustomerID)t
```

subquery in
FROM Clause



Subquery

```
SELECT
CustomerID,
SUM(Sales) TotalSales
FROM Sales.Orders
GROUP BY CustomerID
```



CustomerID	TotalSales
1	110
2	55
3	125
4	90

Main Query

```
SELECT
*,
RANK() OVER (ORDER BY TotalSales DESC) R
FROM
```

**FINAL
RESULT**

CustomerID	TotalSales	R
3	125	1
1	110	2
4	90	3
2	55	4

subquery in **SELECT Clause**

Main Query

SELECT

Column1,

(**SELECT** column **FROM** table1 **WHERE** condition) **AS** alias

FROM table1

subquery

Rules

Only scalar subqueries are allowed to be used

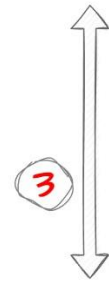
subquery in
**SELECT
clause**

Products		
ProductID	Product	Price
101	Bottle	10
102	Tire	15
103	Socks	20
104	Caps	25
105	Gloves	30

Orders		
OrderID	CustomerID	Sales
1	2	10
2	3	15
3	1	20
4	1	60
5	2	25
6	3	50
7	1	60
8	4	90
9	2	20
10	3	60

Original Query

```
SELECT
  ProductID,
  Product,
  (SELECT COUNT(*)
   FROM Sales.Orders) Total
FROM Sales.Products
```



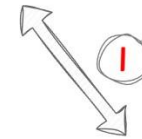
Main Query

```
SELECT
  ProductID,
  Product,
  (10) Total
FROM Sales.Products
```



ProductID	Product	Total
101	Bottle	10
102	Tire	10
103	Socks	10
104	Caps	10
105	Gloves	10

FINAL
RESULT

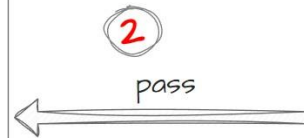


Subquery

```
SELECT COUNT(*)
FROM Sales.Orders
```



COUNT
10



subquery in
WHERE Clause
Comparison Operators

Main Query

```
SELECT column1, column2,...  
  
FROM table1  
  
WHERE column = ( SELECT column FROM table2 WHERE condition )
```

Rules

Only scalar subqueries are allowed to be used

subquery

subquery in
WHERE Clause
Comparison Operators

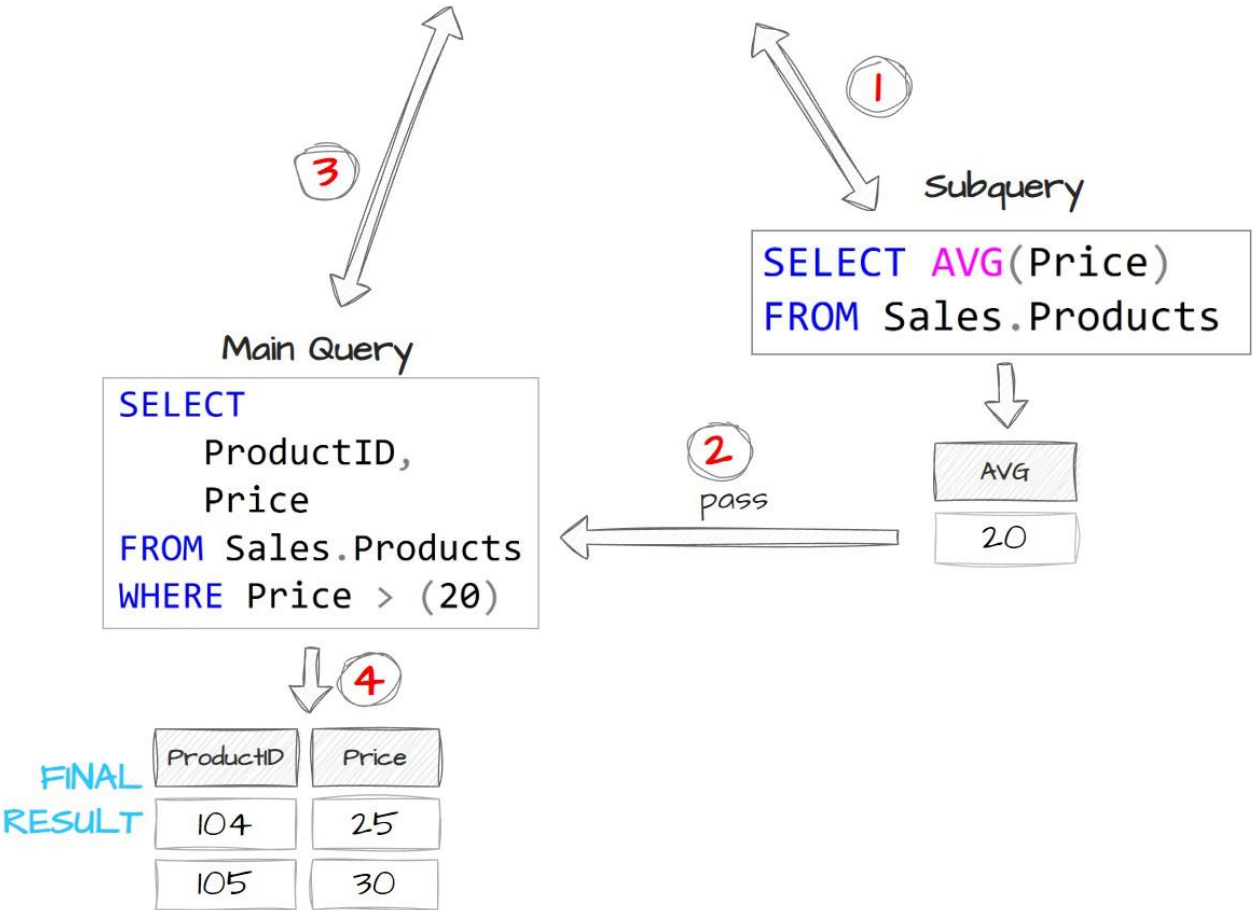
=	Equal	WHERE Sales = (SELECT AVG(Sales) FROM ORDERS)
!= <>	Not Equal	WHERE Sales != (SELECT AVG(Sales) FROM ORDERS)
>	Greater than	WHERE Sales > (SELECT AVG(Sales) FROM ORDERS)
<	Less than	WHERE Sales < (SELECT AVG(Sales) FROM ORDERS)
>=	Greater than or equal to	WHERE Sales >= (SELECT AVG(Sales) FROM ORDERS)
<=	Less than or equal to	WHERE Sales <= (SELECT AVG(Sales) FROM ORDERS)

subquery in
WHERE Clause
Comparison Operators

Products			
ProductID	Product	Category	Price
101	Bottle	Accessories	10
102	Tire	Accessories	15
103	Socks	Clothing	20
104	Caps	Clothing	25
105	Gloves	Clothing	30

Original Query

```
SELECT
    ProductID,
    Price
FROM Sales.Products
WHERE Price >
    (SELECT AVG(Price)
     FROM Sales.Products)
```



subquery in
WHERE Clause
in Operator

Main Query

```
SELECT column1, column2,...
```

```
FROM table1
```

```
WHERE column IN ( SELECT column FROM table2 WHERE condition )
```

subquery

Logical Operators

IN	Checks if a value matches any value in a list	<code>WHERE Sales IN (SELECT ...)</code>
NOT IN	Checks if a value does not matches any value in a list	<code>WHERE Sales NOT IN (SELECT ...)</code>
EXISTS	Checks if subquery returns any rows	<code>WHERE EXISTS (SELECT ...)</code>
NOT EXISTS	Checks if subquery returns no rows	<code>WHERE NOT EXISTS (SELECT ...)</code>
ANY	Returns true if a value matches any value in a list.	<code>WHERE Sales < ANY (SELECT ...)</code>
ALL	Returns true if a value matches all values in a list.	<code>WHERE Sales > ALL (SELECT ...)</code>

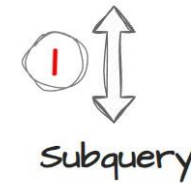
subquery in
WHERE Clause
In Operator

Orders		
OrderID	CustomerID	Sales
1	2	10
2	3	15
3	1	20
4	1	60
5	2	25
6	3	50
7	1	60
8	4	90
9	2	20
10	3	60

Customers				
CustomerID	FirstName	LastName	Country	Score
1	Jossef	Goldberg	Germany	350
2	Kevin	Brown	USA	900
3	Mary	NULL	USA	750
4	Mark	Schwarz	Germany	500
5	Anna	Adams	USA	NULL

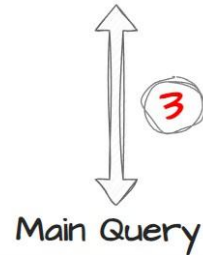
Original Query

```
SELECT
  OrderID,
  CustomerID,
  Sales
FROM Sales.Orders
WHERE CustomerID IN
  (SELECT CustomerID
   FROM Sales.Customers
   WHERE Country = 'Germany')
```



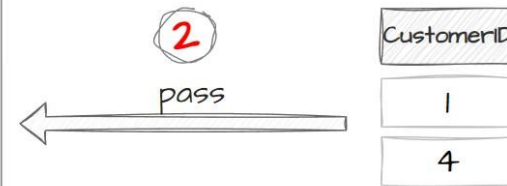
Subquery

```
SELECT CustomerID
FROM Sales.Customers
WHERE Country = 'Germany'
```



Main Query

```
SELECT
  OrderID,
  CustomerID,
  Sales
FROM Sales.Orders
WHERE CustomerID IN (1,4)
```



CustomerID
1
4



FINAL RESULT

OrderID	CustomerID	Sales
3	1	20
4	1	60
7	1	60
8	4	90

subquery in
WHERE Clause
ALL Operators

Main Query

SELECT column1, column2,...

FROM table1

WHERE column < **ALL** (**SELECT** column **FROM** table1 **WHERE** condition)

subquery

subquery in
WHERE Clause
ANY Operator

Main Query

```
SELECT column1, column2,...
```

```
FROM table1
```

```
WHERE column < ANY ( SELECT column FROM table1 WHERE condition )
```

subquery

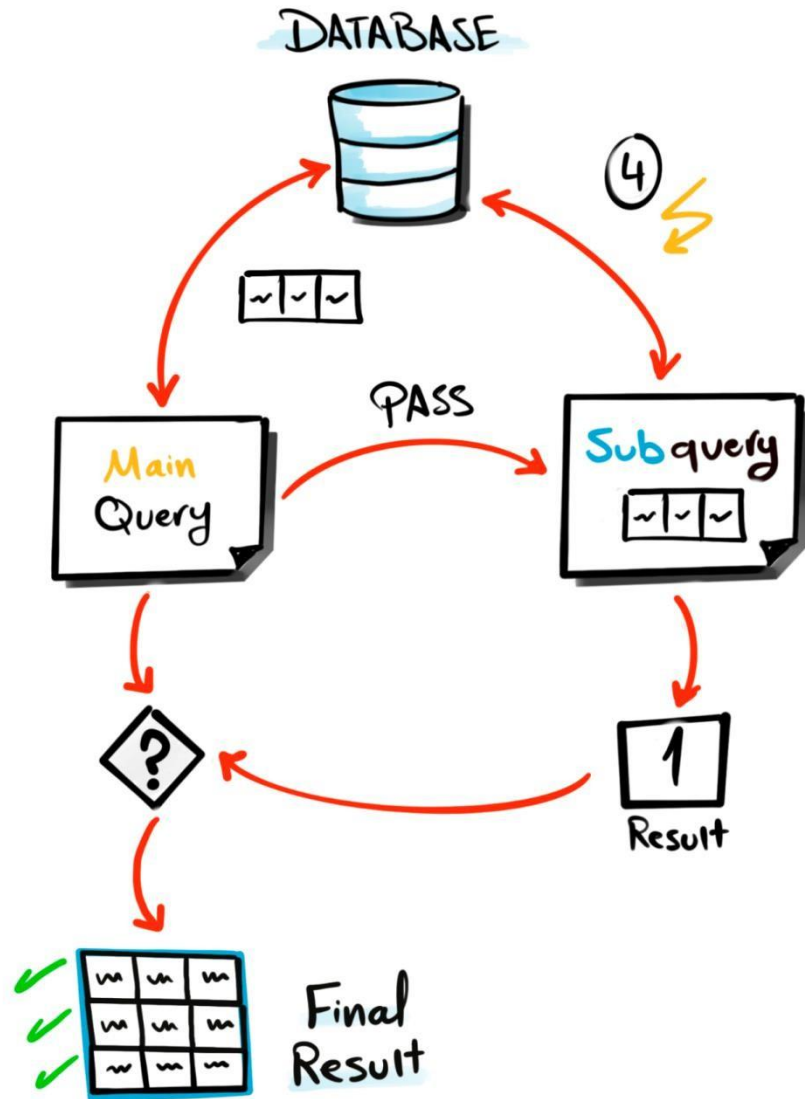
NON-CORRELATED SUBQUERY

A Subquery that can run **independently** from the Main Query

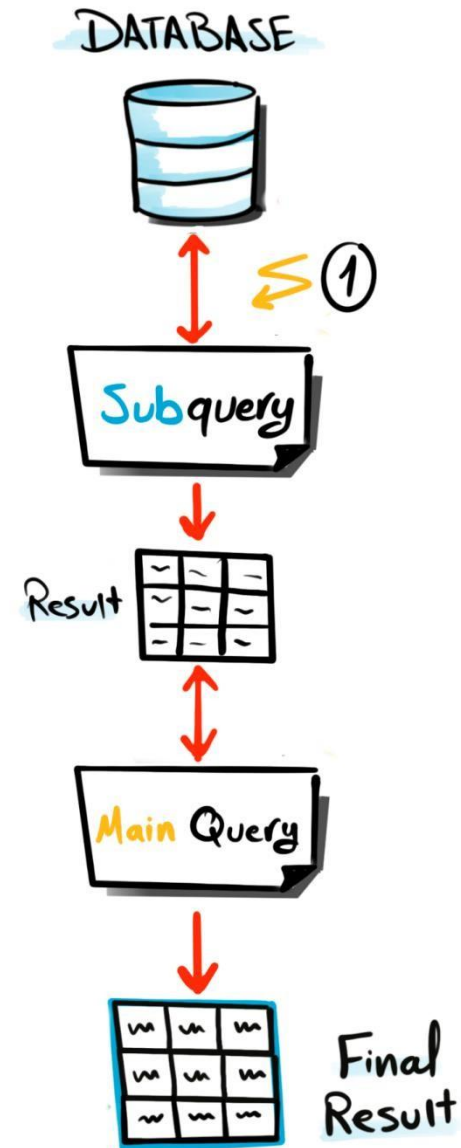
CORRELATED SUBQUERY

A Subquery that **relays** on values from the Main Query

Correlated Subquery



Non-Correlated Subquery



Non-Correlated subquery

Definition

Subquery is **independent** of the main query

Execution

Executed **once** and its result is used by the main query

Can be executed on its Own

easy to use

Easier to read

Performance

Executed **only once** leads to **better** Performance

usage

Static Comparisons, Filtering with Constants

Correlated subquery

Subquery is **dependent** of the main query

Executed for **each row** processed by the main query

Can't be executed on its Own.

Harder to read and more **complex**

Executed multiple times leads to **bad** Performance

Row-by-Row Comparisons, Dynamic Filtering

Correlated subquery in
WHERE Clause
EXISTS Operator

mainQuery

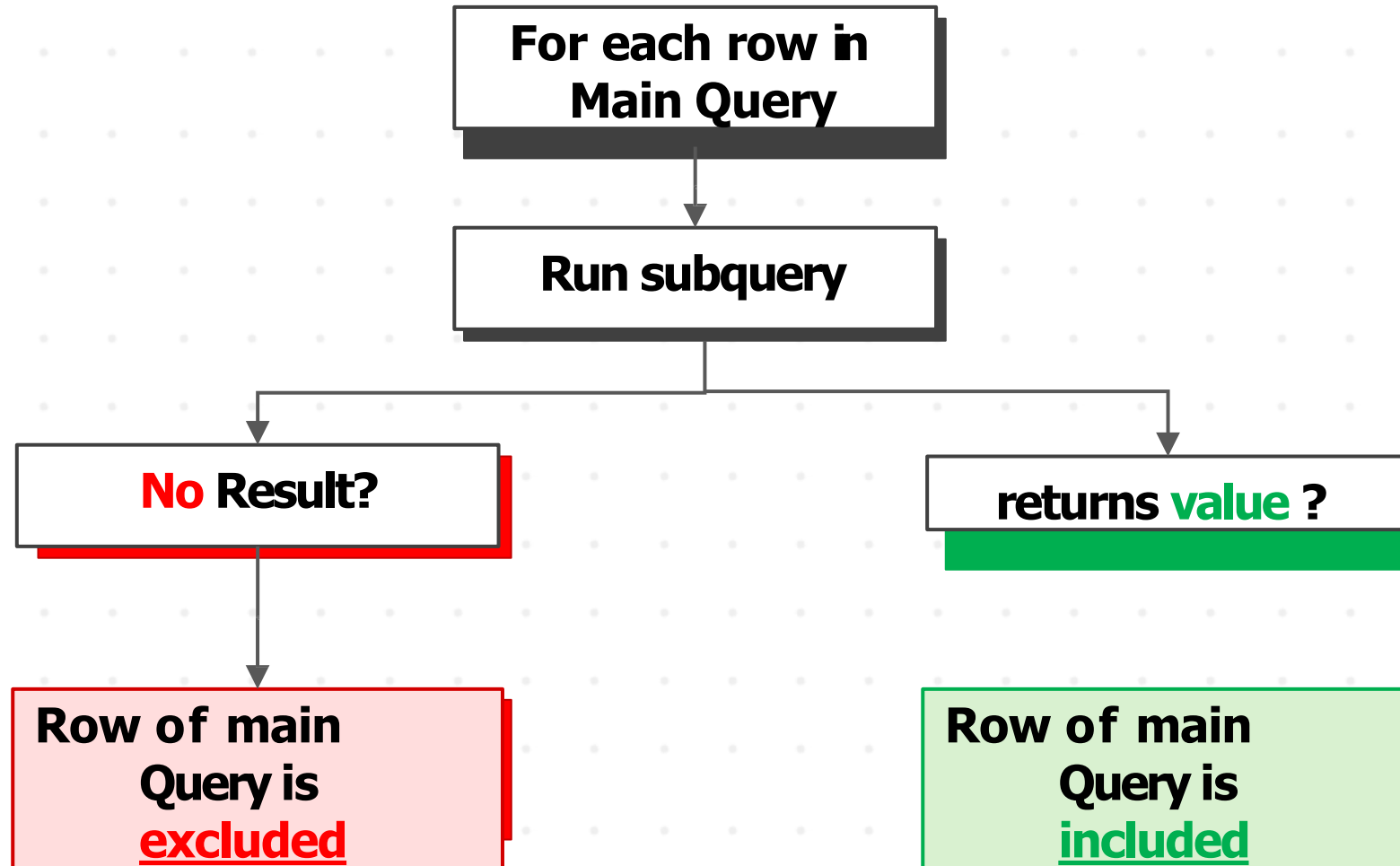
```
SELECT column1, column2,...
```

```
FROM Table2
```

```
WHERE EXISTS ( SELECT 1  
                FROM Table1  
                WHERE Table1.ID = Table2.ID )
```

subquery

How **EXISTS** Works?



JOINS

syntax

```
SELECT o.*  
FROM Orders o  
JOIN Customers c  
ON c.CustomerID = o.CustomerID  
AND c.Country = 'USA'
```

Readability

Not easy to read & maintain

Performance

Fast

Duplicate

May **lead to duplicate**

Best Practices

Useful with **Larg tables**

SUBQUERIES

```
SELECT *  
FROM Orders  
WHERE CustomerID IN  
    (SELECT CustomerID  
     FROM Customers  
     WHERE Country = 'USA')
```

easy to read & maintain

Slow

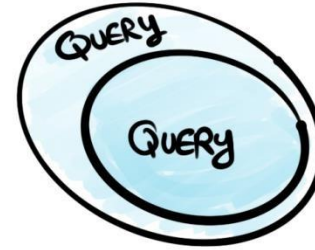
Safer

no risk to have duplicates

Useful with **small tables**

SUBQUERY

Query inside Another Query



Breaks Complex query into smaller, manageable pieces.

USE CASES

- Create Temporary Result Set.
- Prepare Data Before Joining Tables.
- Dynamic & Complex Filtering.
- Check the Existence of Rows From another Table. (EXISTS)
- Row By Row Comparison - Correlated Subquery -